[BUG] qml.qnn.TorchLayer + GPU stopped working in v0.18.0

https://github.com/PennyLaneAI/pennylane/issues/1688

ROW 16

[BUG] qml.qnn.TorchLayer + GPU stopped working in v0.18.0 #1688

Closed #1705

asura opened on Sep 26, 2021

Expected behavior

Learning proceeds without error, as in v0.17.0.

```
Average loss over epoch 1: 0.5028
Average loss over epoch 2: 0.4871
```

Actual behavior

I got an error.

```
RuntimeError: Expected all tensors to be on the same device, but found at least two devices, cuda:0 and (
```

The destination of tensor data seems to be mixed between CPU and GPU.
I checked it out and found the following:

```
input.device=cpu
weight.device=cuda:0
```

Additional information

`requirements.txt` :

```
pennylane == 0.18.0
scikit-learn == 1.0
torch == 1.9.1
```

The sample code is based on the contents of the following forum:
https://discuss.pennylane.ai/t/different-interfaces-different-performances/454

Source code

```
import pennylane as qml
import pennylane.numpy as np
import sklearn.datasets
import torch

n_qubits = 2
dev = qml.device("default.qubit", wires=n_qubits)


@qml.qnode(dev)
def qnode(inputs, weights):
    qml.templates.AngleEmbedding(inputs, wires=range(n_qubits))
    qml.templates.StronglyEntanglingLayers(weights, wires=range(n_qubits))
    return qml.expval(qml.PauliZ(0)), qml.expval(qml.PauliZ(1))


weight_shapes = {"weights": (3, n_qubits, 3)}
qlayer = qml.qnn.TorchLayer(qnode, weight_shapes)

clayer1 = torch.nn.Linear(2, 2)
clayer2 = torch.nn.Linear(2, 2)
softmax = torch.nn.Softmax(dim=1)

device = "cuda"
```

```
clayer2 = torch.nn.Linear(2, 2)
softmax = torch.nn.Softmax(dim=1)

device = "cuda"
model = torch.nn.Sequential(clayer1, qlayer, clayer2, softmax).to(device)

samples = 100
x, y = sklearn.datasets.make_moons(samples)
y_hot = np.zeros((samples, 2))
y_hot[np.arange(samples), y] = 1

X = torch.tensor(x).float()
Y = torch.tensor(y_hot).float()
X, Y = X.to(device), Y.to(device)

epochs = 2
batch_size = 5
batches = samples / batch_size

data_loader = torch.utils.data.DataLoader(
    list(zip(X, Y)),
    batch_size=batch_size,
    shuffle=True,
    drop_last=True
)

opt = torch.optim.SGD(model.parameters(), lr=0.5)
loss = torch.nn.L1Loss()

for epoch in range(epochs):

    running_loss = 0

    for x, y in data_loader:
        opt.zero_grad()
        loss_evaluated = loss(model(x), y)
        loss_evaluated.backward()
        opt.step()
        running_loss += loss_evaluated

    avg_loss = running_loss / batches
    print("Average loss over epoch {}: {:.4f}".format(epoch + 1, avg_loss))
```

Tracebacks

```
Traceback (most recent call last):
  File "ex.py", line 56, in <module>
    loss_evaluated = loss(model(x), y)
  File "(snip)/venv/lib/python3.8/site-packages/torch/nn/modules/module.py", line 1051, in _call_impl
    return forward_call(*input, **kwargs)
  File "(snip)/venv/lib/python3.8/site-packages/torch/nn/modules/container.py", line 139, in forward
    input = module(input)
  File "(snip)/venv/lib/python3.8/site-packages/torch/nn/modules/module.py", line 1051, in _call_impl
    return forward_call(*input, **kwargs)
  File "(snip)/venv/lib/python3.8/site-packages/torch/nn/modules/linear.py", line 96, in forward
    return F.linear(input, self.weight, self.bias)
  File "(snip)/venv/lib/python3.8/site-packages/torch/nn/functional.py", line 1847, in linear
    return torch._C._nn.linear(input, weight, bias)
```

System information

```
WARNING: pip is being invoked by an old script wrapper. This will fail in a future version of pip.
Please see https://github.com/pypa/pip/issues/5599 for advice on fixing the underlying issue.
To avoid this problem you can invoke Python with '-m pip' instead of running pip directly.
Name: PennyLane
Version: 0.18.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author: None
Author-email: None
License: Apache License 2.0
Location: /home/asura/git/qml_ng/venv/lib/python3.8/site-packages
Requires: cachetools, autoray, numpy, appdirs, pennylane-lightning, toml, autograd, semantic-version, scipy, r
Required-by: PennyLane-Lightning
Platform info:           Linux-5.10.16.3-microsoft-standard-WSL2-x86_64-with-glibc2.29
```

System information

```
WARNING: pip is being invoked by an old script wrapper. This will fail in a future version of pip.
Please see https://github.com/pypa/pip/issues/5599 for advice on fixing the underlying issue.
To avoid this problem you can invoke Python with '-m pip' instead of running pip directly.
Name: PennyLane
Version: 0.18.0
Summary: PennyLane is a Python quantum machine learning library by Xanadu Inc.
Home-page: https://github.com/XanaduAI/pennylane
Author: None
Author-email: None
License: Apache License 2.0
Location: /home/asura/git/qml_ng/venv/lib/python3.8/site-packages
Requires: cachetools, autoray, numpy, appdirs, pennylane-lightning, toml, autograd, semantic-version, scipy, r
Required-by: PennyLane-Lightning
Platform info:           Linux-5.10.16.3-microsoft-standard-WSL2-x86_64-with-glibc2.29
Python version:          3.8.10
Numpy version:           1.21.2
Scipy version:           1.7.1
Installed devices:
- default.gaussian (PennyLane-0.18.0)
- default.mixed (PennyLane-0.18.0)
- default.qubit (PennyLane-0.18.0)
- default.qubit.autograd (PennyLane-0.18.0)
- default.qubit.jax (PennyLane-0.18.0)
- default.qubit.tf (PennyLane-0.18.0)
- default.qubit.torch (PennyLane-0.18.0)
- default.tensor (PennyLane-0.18.0)
- default.tensor.tf (PennyLane-0.18.0)
- lightning.qubit (PennyLane-Lightning-0.18.0)
```

- [x] I have searched exisisting GitHub issues to make sure the issue does not already exist.

asura added bug on Sep 26, 2021

github-actions assigned antalszava on Sep 26, 2021

mlxd on Sep 27, 2021 — Member

Hi **@asura** thank you for reporting this. I can confirm the same issue locally. We will look into this and get back to you.

antalszava mentioned this in 2 pull requests on Sep 29, 2021

- Update `torch.rst` #1704
- Fix GPU usage with `default.qubit.torch` for `qml.qnn.TorchLayer` #1705

josh146 closed this as completed in #1705 on Oct 8, 2021

antalszava commented on Sep 29, 2021 • edited Contributor ···

Context

1.

In PennyLane v0.18.0, the `default.qubit.torch` device was added. This device is used under the hood by QNodes for backpropagation with Torch tensors. The QNode, however, doesn't have a logic for checking what the `torch_device` keyword argument should be when creating a backdrop device and instantiates `default.qubit.torch` with default arguments (`torch_device='cpu'`). That is, there is no opportunity to specify using the GPU.

Therefore, there may be issues with passing Torch tensors using CUDA as QNode arguments (see #1688). What we would like to have in such cases is equivalent to `dev = qml.device('default.qubit.torch', wires=3, torch_device='cuda')`.

2.

Further to 1., the `default.qubit.torch` device uses the internal methods of `DefaultQubit` in many cases. These methods use the `_asarray` method. One detail of `_asarray` is that it doesn't consider the Torch device that's used, hence computations might shift from the GPU to CPU.

Changes

1. Changes the QNode to specify `torch_device='cuda'` internally when creating a `dev = qml.device('default.qubit.torch', wires=3)` device for backpropagation.
2. Changes the `_asarray` method to always put the output on the Torch device specified internally;

Related issues

Closes #1688.